



Introduction to Linux OS

By Mohamed Gamal



Analyzing and Sorting Logs

In Linux, system logs are stored in the **/var/log** directory, which contains log files for various system activities, applications, services, and security. Analyzing and sorting logs in Linux is a crucial step for troubleshooting, monitoring, and understanding the behavior of the system.

```
[MG@centos9 ~]$ ls -lhF /var/log
total 5.2M
drwxr-xr-x. 2 root root 4.0K Nov 29 14:11 anaconda/
drwx----- 2 root root 23 Aug 1 15:42 audit/
-rw----- 1 root root 0 Dec 6 00:00 boot.log
drwxr-x--- 2 chrony chrony 6 Nov 11 11:44 chrony/
-rw----- 1 root root 21K Dec 6 00:01 cron
drwxr-xr-x. 2 lp sys 57 Oct 1 16:25 cups/
-rw-r----- 1 root root 50 Dec 4 11:57 firewalld
drwx--x--x. 2 root gdm 6 Jul 26 21:36 gdm/
-rw-r--r-- 1 root root 1.1K Dec 5 23:21 hawkey.log
-rw-rw-r-- 1 root utmp 286K Dec 5 19:10 lastlog
-rw----- 1 root root 0 Nov 29 14:04 maillog
-rw----- 1 root root 4.4M Dec 6 00:16 messages
drwx----- 2 root root 6 Nov 29 14:02 private/
drwx----- 3 root root 17 Oct 22 13:20 samba/
-rw----- 1 root root 94K Dec 6 00:15 secure
drwx----- 2 root root 6 Aug 10 2021 speech-dispatcher/
-rw----- 1 root root 0 Nov 29 14:04 spooler
drwxr-x--- 2 sssd sssd 26 Oct 21 16:17 sssd/
-rw----- 1 root root 0 Nov 29 14:02 tallylog
drwxr-xr-x. 2 root root 23 Aug 8 00:36 tuned/
-rw-rw-r-- 1 root utmp 30K Dec 5 19:10 wtmp
```

Log File	Purpose
/var/log/messages	Most syslog messages.
/var/log/secure	Log file for security and authentication-related messages and errors.
/var/log/maillog	Log file with main server-related messages.
/var/log/boot.log	Messages related to system start up.
...	...

```
$ df -h # report file system disk space usage
```

```
$ cat /var/log/messages
$ less /var/log/boot.log
$ tail -f /var/log/messages
$ grep "error" /var/log/messages
$ sort /var/log/messages
$ awk '{print $1, $2, $3}' /var/log/ messages
```



Scheduling Jobs

Cron is a time-based job scheduling service that automates tasks or performs routine maintenance. It checks for scheduled jobs every minute and executes them accordingly.

- **cron:** the service responsible for running scheduled jobs, typically started when the system boots.
- **Crontab (Cron Table):** a program used to create, read, update, and delete job schedules. The crontab file defines when and what commands to run, with five time specification fields (minutes, hour, day of the month, month of the year, and day of the week), followed by the command to execute.

Crontab Format

```
* * * * * command
| | | | |
| | | | +-- Day of the Week (0-6)
| | | +---- Month of the Year (1-12)
| | +----- Day of the Month (1-31)
| +----- Hour (0-23)
+----- Minute (0-59)
```

Commands	
\$ crontab -e	Edit (e) the crontab file for the current user.
\$ crontab -l	List (l) current user's crontab entries.
\$ crontab -r	Remove (r) the current user's crontab file.

Examples ([Crontab Generator](#)):

```
# Run every Monday at 07:00.
0 7 * * 1 /opt/sales/bin/weekly-report
```

```
0 7 * * 1 /opt/sales/bin/weekly-report
| | | | |
| | | | +-- Day of the Week (0-6)
| | | +---- Month of the Year (1-12)
| | +----- Day of the Month (1-31)
| +----- Hour (0-23)
+----- Minute (0-59)
```

```
0 0 * * 3 <exec_file> # Cron job every Wednesday at midnight
* /30 * * * ls -l # Cron job every half hour
0 0 * * 1-5 # Run on weekdays only at midnight
```

Example
<pre>\$ crontab -e # open crontab file * * * * * rm /home/mg/.local/share/Trash/files/* \$ sudo tail -f /var/log/messages</pre>

Once you write and save a crontab entry using **\$ crontab -e**, the cron daemon (cron) automatically schedules the job based on the specified timing.

```
$ sudo systemctl status crond
```



Package Management on CentOS, Fedora, and RedHat-based Distributions (Installing Software)

A **package** in Linux is a collection of files that make up an application, along with metadata such as the app's description, version, and dependencies. To install or remove packages, superuser privileges are required.

A **package manager** is used to install, upgrade, and remove packages, while also handling dependencies and tracking installed packages and their versions.

1) Using yum Package Manager

The **yum (Yellowdog Updater, Modified)** command is a package management tool for RPM-based distributions like CentOS, Fedora, and RedHat.



Search for a package

```
$ yum search <package name>
```

```
$ yum search inkscape
```

Install a package

```
$ sudo yum install <package name>
```

```
$ sudo yum install inkscape
```

Remove a package

```
$ sudo yum remove <package name>
```

```
$ sudo yum remove inkscape
```

Get information about a package

```
$ yum info <package name>
```

```
$ yum info inkscape
```

List all installed/available Packages

```
$ sudo yum list available|installed
```

```
$ sudo yum list installed
```

Check for available updates

```
$ sudo yum check-update
```

```
$ sudo yum check-update
```

Update a Specific Package

```
$ sudo yum update <package name>
```

```
$ sudo yum update nginx
```

Update All System Packages

```
$ sudo yum update
```

```
$ sudo yum update
```



2) Using dnf Package Manager

The **dnf** (**D**andified **YUM**) package manager is an improved, modern version of **yum**, designed to address some limitations of **yum** while maintaining its core functionality.

3) Using rpm Package Manager (Low-Level)

The **rpm** (**R**ed**H**at **P**ackage **M**anager) command can also be used to manage packages, such as installing a package from a file, and listing files belonging to a package, ..., etc.

Search for a Package
<code>\$ rpm -qi <package name></code>

```
$ rpm -qi SpiderOak
```

Get information about a Package
<code>\$ rpm -qi <package name></code>

```
$ rpm -qi SpiderOak
```

Download an RPM Package
<code>\$ dnf download <package name></code>

```
$ dnf download nano
```

Install a Package
<code>\$ rpm -ivh <package.rpm></code>

```
$ sudo rpm -ivh nano-*.rpm
```

List all installed Packages
<code>\$ rpm -qa</code>

```
$ rpm -qa | sort | head
```

Update a Specific Package
<code>\$ rpm -Uvh <package.rpm></code>

```
$ sudo rpm -Uvh update nginx
```

Remove a package
<code>\$ rpm -e <package name></code>

```
$ sudo rpm -e SpiderOak
```

List all files belonging to a package
<code>\$ rpm -ql <package name></code>

```
$ rpm -ql inkscape
```

The **RPM** package manager itself does not handle package updates or dependency resolution. It is a low-level package manager that installs, removes, and queries individual .rpm packages but does not manage repositories or system-wide updates.

For updating packages, you need to use **DNF** (or **YUM**). However, if you still want to use RPM, you would need to manually download and install updated packages.



--- yum vs. dnf vs. rpm ---

Feature	yum (Legacy)	dnf (Modern)	rpm (low-level)
Default in	CentOS 7 and older	CentOS 8, 9 (Stream), RHEL 8+	All RHEL-based distros
Performance	Slower, higher memory usage	Faster, lower memory usage	Very fast, but lacks dependency resolution
Dependency Resolution	Uses older algorithms	More efficient dependency solving	No automatic dependency resolution
Plugin System	Relies on plugins for many features	Built-in support for advanced features	No plugin support
Package Caching	Stores metadata in /var/cache/yum/	Uses sqlite3 for faster processing	No package caching
Parallel Downloads	Not supported	Supported (improves speed)	N/A
Command Syntax	\$ yum install package	\$ dnf install package (same syntax)	\$ rpm -ivh package.rpm

Example: <https://code.visualstudio.com/Download>

